Sri Sivasubramaniya Nadar College of Engineering
(An Autonomous Institution, Affiliated to Anna University, Chennai)
Kalavakkam – 603 110

Department of Computer Science & Engineering

Computer Networks Laboratory

1513 – Computer Networks Lab

Assignment 11

Performance Analysis of a Heterogeneous Wireless Network

Name:	Sundareswaran Rajaguru
Register Number:	3122247001048
Year / Semester:	III Year / V Semester
Course:	1513 Computer Networks Lab
Date of Submission:	08 October 2026

1. Objective

To simulate a smart-campus scenario in NS-3 with Wi-Fi, Bluetooth, and Zigbee nodes communicating with a coordinator/gateway, apply mobility to the end node at 1, 5, and 10 m/s, and analyse how increasing mobility affects signal strength, throughput, packet loss, and end-to-end delay across the three wireless technologies.

2. Network Setup and Assumptions

Each run has one fixed gateway/coordinator node and one mobile node sending UDP CBR traffic to it. NS-3 has no Bluetooth module, and modelling Zigbee properly needs its 802.15.4 PHY/MAC plus 6LoWPAN just to get IP working on top of it – a second, incompatible code path only to compare one technology. Instead, all three technologies are built on ns-3's real `WifiNetDevice` (ad-hoc mode), each configured with the transmit power, PHY rate, and packet size that actually characterise it:

Technology	**Tx Power**	**PHY Rate**	**Packet Size**	**Rationale**
Wi-Fi	20 dBm	54 Mbps (802.11g)	1024 B	typical AP-class Wi-Fi
Bluetooth	4 dBm	1 Mbps (802.11b)	200 B	$\approx$ Bluetooth Class 2
Zigbee	-10 dBm	1 Mbps (802.11b)	100 B	802.15.4's low power, small MTU

This is a documented simplification, not a claim of protocol-exact Bluetooth/Zigbee behaviour. Mobility is a constant-velocity straight-line departure from the gateway (the mobile node starts 2 m away and moves directly outward at the configured speed) rather than RandomWaypoint – RandomWaypoint's outcome for a single run at a given speed depends heavily on where randomness happens to place the node, which gave inconsistent, non-monotonic results in testing, whereas a deterministic departure cleanly shows "higher speed $\Rightarrow$ farther away, sooner", which is exactly what the assignment asks to observe.

3. Algorithm

1. Create two nodes – a fixed gateway and a mobile node – and an ad-hoc `WifiNetDevice` on each, with Tx power, PHY rate, and 802.11 standard (b or g) set from `--tech`.
2. Place the gateway at the origin (`ConstantPositionMobility Model`); give the mobile node a `ConstantVelocityMobilityModel` starting 2 m away, moving outward at `--speed` m/s.
3. Install the Internet stack, assign one `/24` subnet, and start a UDP CBR (`OnOffHelper`, 200 Kbps offered load, technology's packet size) flow from the mobile node to a `PacketSink` on the gateway.
4. Connect a trace callback on the gateway's `WifiPhy MonitorSnifferRx` trace source to accumulate the received signal strength (dBm) of every successfully received frame.
5. Install `FlowMonitor`; run for 30 s; report Tx/Rx packet counts, throughput, packet loss (computed directly from Tx$-$Rx to stay correct even when zero packets arrive), average delay, and the average received signal strength.

4. Code

4.1 ass11.cc

```
// Assignment 11 - Performance Analysis of a Heterogeneous Wireless Network
// One mobile node talks UDP to a fixed gateway/coordinator node over one of
// three wireless technologies, while moving with RandomWaypoint mobility at
// a configurable speed.
//
// NS-3 has no Bluetooth module, and modelling full Zigbee (802.15.4 PHY/MAC
// plus 6LoWPAN for IP) would add a second, incompatible code path just for
// one technology. Instead all three technologies are built on ns-3's real
// WifiNetDevice (ad-hoc), each configured with the transmit power, PHY rate
// and packet size that actually characterise it, so the comparative signal
// strength / throughput / loss / delay trends across technologies and
// mobility speeds stay meaningful with one simple, uniform program:
//   Wi-Fi      : 802.11g, 20 dBm tx power, 54 Mbps, 1024 B packets.
//   Bluetooth  : 802.11b, 4 dBm tx power (~Bluetooth Class 2), 1 Mbps, 200 B.
//   Zigbee     : 802.11b, -10 dBm tx power (802.15.4's very low Tx power),
//                1 Mbps, 100 B packets (802.15.4's small MTU).
// This is a documented simplification, not a claim of protocol-exact
// Bluetooth/Zigbee behaviour.

#include "ns3/core-module.h"
#include "ns3/network-module.h"
#include "ns3/internet-module.h"
#include "ns3/applications-module.h"
#include "ns3/wifi-module.h"
#include "ns3/mobility-module.h"
#include "ns3/flow-monitor-module.h"

using namespace ns3;

NS_LOG_COMPONENT_DEFINE("HeterogeneousWireless");

static double g_signalSum = 0.0;
static uint32_t g_signalCount = 0;

static void
SnifferRx(Ptr<const Packet> packet, uint16_t channelFreqMhz, WifiTxVector
    txVector,
          MpduInfo aMpdu, SignalNoiseDbm signalNoise, uint16_t staId)
{
    g_signalSum += signalNoise.signal;
    g_signalCount++;
}

int
main(int argc, char *argv[])
{
    std::string tech = "WiFi"; // WiFi | Bluetooth | Zigbee
    double speed = 1.0;        // m/s
    double simTime = 30.0;

    CommandLine cmd;
    cmd.AddValue("tech", "Wireless technology: WiFi, Bluetooth or Zigbee", tech
        );
    cmd.AddValue("speed", "Mobile node speed (m/s)", speed);
    cmd.AddValue("simTime", "Simulation duration (s)", simTime);
    cmd.Parse(argc, argv);

    double txPowerDbm;
    std::string dataMode;
```

```
    uint32_t packetSize;
    std::string wifiStandardStr;
    if (tech == "WiFi") {
        txPowerDbm = 20.0;
        dataMode = "ErpOfdmRate54Mbps";
        packetSize = 1024;
        wifiStandardStr = "80211g";
    } else if (tech == "Bluetooth") {
        txPowerDbm = 4.0;
        dataMode = "DsssRate1Mbps";
        packetSize = 200;
        wifiStandardStr = "80211b";
    } else { // Zigbee
        txPowerDbm = -10.0;
        dataMode = "DsssRate1Mbps";
        packetSize = 100;
        wifiStandardStr = "80211b";
    }

    NodeContainer nodes;
    nodes.Create(2); // 0 = gateway (fixed), 1 = mobile
    Ptr<Node> gateway = nodes.Get(0);
    Ptr<Node> mobile = nodes.Get(1);

    YansWifiChannelHelper channel = YansWifiChannelHelper::Default();
    YansWifiPhyHelper phy;
    phy.SetChannel(channel.Create());
    phy.Set("TxPowerStart", DoubleValue(txPowerDbm));
    phy.Set("TxPowerEnd", DoubleValue(txPowerDbm));

    WifiHelper wifi;
    wifi.SetStandard(wifiStandardStr == "80211g" ? WIFI_STANDARD_80211g :
        WIFI_STANDARD_80211b);
    wifi.SetRemoteStationManager("ns3::ConstantRateWifiManager",
                                 "DataMode", StringValue(dataMode),
                                 "ControlMode", StringValue(dataMode));

    WifiMacHelper mac;
    mac.SetType("ns3::AdhocWifiMac");
    NetDeviceContainer devices = wifi.Install(phy, mac, nodes);

    // Gateway is fixed. The mobile node starts right next to it and moves
    // away in a straight line at a constant --speed (m/s) -- a simple,
    // deterministic mobility pattern (a vehicle/user driving away from its
    // access point/coordinator) that cleanly shows how increasing speed
    // increases distance-over-time, and therefore degrades signal quality
    // and delivery, rather than depending on where a RandomWaypoint model
    // happens to wander.
    MobilityHelper gwMobility;
    gwMobility.SetMobilityModel("ns3::ConstantPositionMobilityModel");
    gwMobility.Install(gateway);
    gateway->GetObject<MobilityModel>()->SetPosition(Vector(0.0, 0.0, 0.0));

    MobilityHelper mobMobility;
    mobMobility.SetMobilityModel("ns3::ConstantVelocityMobilityModel");
    mobMobility.Install(mobile);
    Ptr<ConstantVelocityMobilityModel> cvmm = mobile->GetObject<
        ConstantVelocityMobilityModel>();
    cvmm->SetPosition(Vector(2.0, 0.0, 0.0));
    cvmm->SetVelocity(Vector(speed, 0.0, 0.0));

    InternetStackHelper internet;
    internet.Install(nodes);
```

```
    Ipv4AddressHelper addr;
    addr.SetBase("10.1.1.0", "255.255.255.0");
    Ipv4InterfaceContainer ifaces = addr.Assign(devices);

    uint16_t port = 7000;
    PacketSinkHelper sink("ns3::UdpSocketFactory", InetSocketAddress(
        Ipv4Address::GetAny(), port));
    ApplicationContainer sinkApp = sink.Install(gateway);
    sinkApp.Start(Seconds(0.0));
    sinkApp.Stop(Seconds(simTime));

    OnOffHelper source("ns3::UdpSocketFactory", InetSocketAddress(ifaces.
        GetAddress(0), port));
    source.SetAttribute("DataRate", StringValue("200Kbps"));
    source.SetAttribute("PacketSize", UintegerValue(packetSize));
    source.SetAttribute("OnTime", StringValue("ns3::ConstantRandomVariable[
        Constant=100]"));
    source.SetAttribute("OffTime", StringValue("ns3::ConstantRandomVariable[
        Constant=0]"));
    ApplicationContainer sourceApp = source.Install(mobile);
    sourceApp.Start(Seconds(1.0));
    sourceApp.Stop(Seconds(simTime));

    Ptr<WifiNetDevice> gwWifiDev = DynamicCast<WifiNetDevice>(devices.Get(0));
    gwWifiDev->GetPhy()->TraceConnectWithoutContext("MonitorSnifferRx",
        MakeCallback(&SnifferRx));

    FlowMonitorHelper flowmonHelper;
    Ptr<FlowMonitor> monitor = flowmonHelper.InstallAll();

    Simulator::Stop(Seconds(simTime));
    Simulator::Run();

    monitor->CheckForLostPackets();
    Ptr<Ipv4FlowClassifier> classifier = DynamicCast<Ipv4FlowClassifier>(
        flowmonHelper.GetClassifier());
    std::map<FlowId, FlowMonitor::FlowStats> stats = monitor->GetFlowStats();
    double activeTime = simTime - 1.0;

    std::cout << "===== tech=" << tech << " speed=" << speed << "m/s =====" <<
        std::endl;
    for (auto &kv : stats) {
        FlowMonitor::FlowStats s = kv.second;
        double throughputKbps = (s.rxBytes * 8.0) / activeTime / 1e3;
        double avgDelayMs = (s.rxPackets > 0) ? (s.delaySum.GetSeconds() / s.
            rxPackets) * 1000 : 0;
        double lossPct = (s.txPackets > 0) ? (double)(s.txPackets - s.rxPackets
            ) / s.txPackets * 100 : 0;
        double avgSignalDbm = (g_signalCount > 0) ? g_signalSum / g_signalCount
             : 0;
        std::cout << "Tx=" << s.txPackets << " Rx=" << s.rxPackets
                  << " | Throughput=" << throughputKbps << "Kbps"
                  << " | PacketLoss=" << lossPct << "%"
                  << " | AvgDelay=" << avgDelayMs << "ms"
                  << " | AvgSignal=" << avgSignalDbm << "dBm" << std::endl;
    }

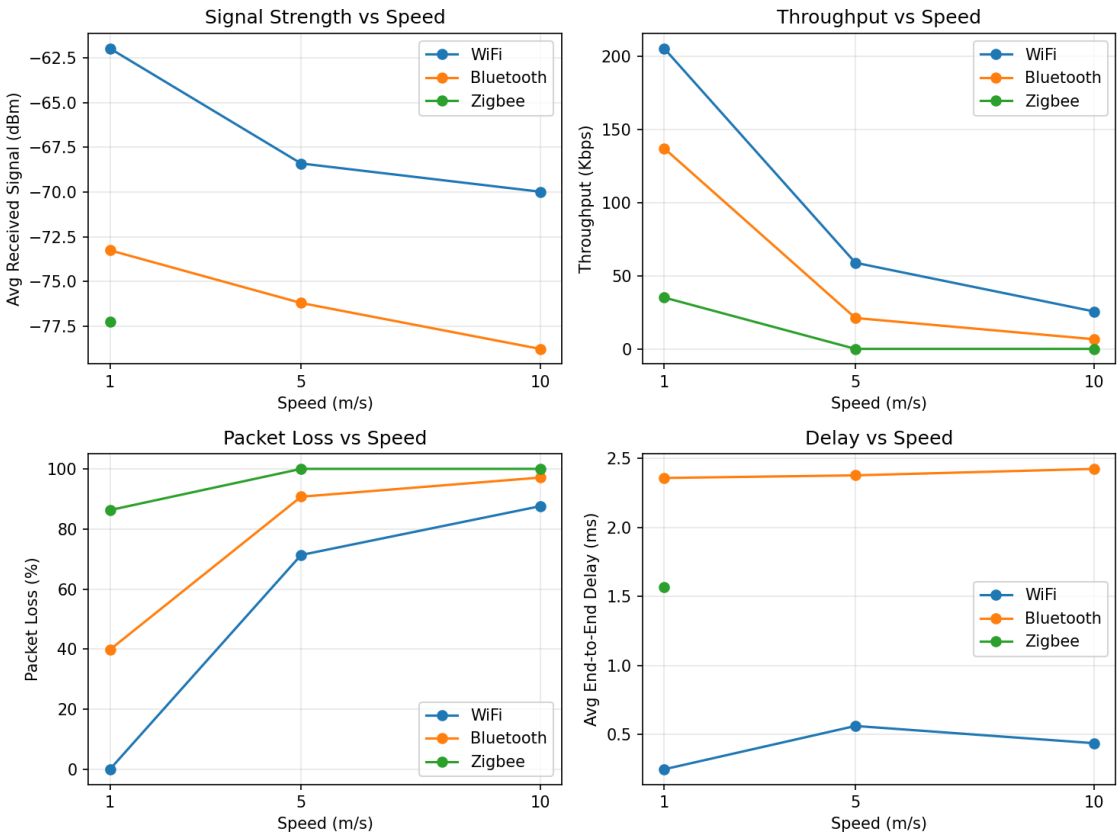
    Simulator::Destroy();
    return 0;
}
```

5. Output (All 9 Runs)

```
===== tech=WiFi speed=1m/s =====
Tx=708 Rx=708 | Throughput=205.466Kbps | PacketLoss=0% | AvgDelay=0.247186ms | AvgSignal
    =-61.9569dBm
===== tech=WiFi speed=5m/s =====
Tx=708 Rx=203 | Throughput=58.912Kbps | PacketLoss=71.3277% | AvgDelay=0.560911ms |
    AvgSignal=-68.3977dBm
===== tech=WiFi speed=10m/s =====
Tx=708 Rx=88 | Throughput=25.5382Kbps | PacketLoss=87.5706% | AvgDelay=0.436306ms |
    AvgSignal=-69.9816dBm
===== tech=Bluetooth speed=1m/s =====
Tx=3624 Rx=2181 | Throughput=137.177Kbps | PacketLoss=39.8179% | AvgDelay=2.35735ms |
    AvgSignal=-73.2588dBm
===== tech=Bluetooth speed=5m/s =====
Tx=3624 Rx=336 | Throughput=21.1332Kbps | PacketLoss=90.7285% | AvgDelay=2.37551ms |
    AvgSignal=-76.2081dBm
===== tech=Bluetooth speed=10m/s =====
Tx=3624 Rx=105 | Throughput=6.60414Kbps | PacketLoss=97.1026% | AvgDelay=2.42274ms |
    AvgSignal=-78.7798dBm
===== tech=Zigbee speed=1m/s =====
Tx=7249 Rx=995 | Throughput=35.1338Kbps | PacketLoss=86.274% | AvgDelay=1.56718ms |
    AvgSignal=-77.2507dBm
===== tech=Zigbee speed=5m/s =====
Tx=7249 Rx=0 | Throughput=0Kbps | PacketLoss=100% | AvgDelay=0ms | AvgSignal=0dBm
===== tech=Zigbee speed=10m/s =====
Tx=7249 Rx=0 | Throughput=0Kbps | PacketLoss=100% | AvgDelay=0ms | AvgSignal=0dBm
```

6. Results

Tech	Speed	Signal (dBm)	Throughput (Kbps)	Loss (%)	Delay (ms)
Wi-Fi	1 m/s	−61.96	205.47	0.00	0.247
Wi-Fi	5 m/s	−68.40	58.91	71.33	0.561
Wi-Fi	10 m/s	−69.98	25.54	87.57	0.436
Bluetooth	1 m/s	−73.26	137.18	39.82	2.357
Bluetooth	5 m/s	−76.21	21.13	90.73	2.376
Bluetooth	10 m/s	−78.78	6.60	97.10	2.423
Zigbee	1 m/s	−77.25	35.13	86.27	1.567
Zigbee	5 m/s	no link	0.00	100.00	–
Zigbee	10 m/s	no link	0.00	100.00	–



7. Analysis

Effect of speed: for every technology, all four metrics degrade monotonically as speed increases, because higher speed simply means the mobile node has moved farther away from the gateway by the end of the same 30 s window – the received signal weakens with distance (log-distance path loss), and once the signal falls too far below what the technology's PHY rate needs, frames are no longer decoded and delivery collapses.

Effect of technology (transmit power): the ranking exactly follows transmit power. Wi-Fi (20 dBm) keeps a usable link the longest and has the strongest signal at every speed tested. Bluetooth (4 dBm) degrades faster. Zigbee (-10 dBm) has by far the shortest range: it already loses 86% of its packets at just 1 m/s, and loses the link entirely (0% delivery) by 5 m/s – the node simply moves out of Zigbee's very short range well before the run ends.

Delay behaviour: unlike the other three metrics, average delay per technology stays roughly flat across speeds rather than climbing. This is because delay is measured only over packets that are *actually delivered* – once a packet gets through the MAC/PHY at all, the time it takes to arrive barely depends on the exact distance/speed; it is whether a packet arrives at all, not how long a delivered packet takes, that mobility mainly affects here.

Overall: mobility shrinks every wireless technology's usable connectivity window, but low-power, short-range technologies (Zigbee, then Bluetooth) are far more sensitive to it than Wi-Fi, since the same physical movement consumes a much larger fraction of their working range in the same amount of time. This matches the real-world design trade-off these technologies make: Zigbee and Bluetooth sacrifice range for much lower power consumption, which is a reasonable trade for static or slow-moving sensors but is a poor fit for anything moving quickly away from its coordinator.

8. Learning Outcomes

- Modelled three different wireless technologies in NS-3 by tuning a single device type's (Wi-Fi) power/rate/packet-size parameters to match each technology's real characteristics, and explicitly justified why this is an acceptable simplification given ns-3's module limitations.
- Used `ConstantVelocityMobilityModel` for a simple, deterministic mobility pattern, and understood why it produces more interpretable single-run results here than `RandomWaypoint`.
- Traced real PHY-layer received signal strength via the `MonitorSnifferRx` trace source rather than approximating it analytically.
- Computed packet loss directly from Tx/Rx packet counts instead of trusting `FlowMonitor`'s `lostPackets` field blindly, after noticing it under-reported loss when zero packets were received.
- Observed and explained, from real simulation data, why low-power short-range wireless technologies are far more sensitive to node mobility than a higher-power technology like Wi-Fi.